

UNIVERSIDADE FEDERAL DE LAVRAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Curso: *Ciência da Computação*

GCC130 – Compiladores

Professor: *Ricardo Terra*

Pontuação: *40 pontos (3 questões)*

TP

Data: *Veja etapa*

INFORMAÇÕES SOBRE TP:

1. Atividades entregues **após o prazo** terão penalização na nota. Logo, fiquem atentos à data de entrega.
2. Cópias (total ou parcial) serão penalizadas com **nota zero** em todos os trabalhos.
3. A atividade é grupo de **exatamente** 2 (dois) alunos.
4. O trabalho deve ser entregue pelo *Campos Virtual*.
5. Enviar apenas um arquivo compactado no formato *zip* conforme definições da seção “Entrega e execução”.

O objetivo do trabalho prático é **projetar e desenvolver** um compilador para uma mini linguagem didática. O trabalho está dividido em 3 etapas, conforme descrição a seguir.

A definição da linguagem também faz parte do trabalho. Os elementos explicitamente estabelecidos neste enunciado constituem os **requisitos mínimos** da linguagem. Aspectos não especificados — como detalhes da sintaxe concreta, organização das produções da gramática, forma das chamadas às funções predefinidas e estratégias de implementação — são **decisões de projeto do grupo**. Essas decisões devem ser coerentes, mantidas ao longo das três etapas e justificadas no relatório.

Aspectos não definidos explicitamente neste enunciado não possuem uma solução única esperada, cabendo ao grupo pesquisar, definir, implementar e justificar uma solução tecnicamente consistente.

Não será fornecida uma gramática completa da linguagem. A definição de uma sintaxe coerente que satisfaça os requisitos estabelecidos faz parte do trabalho.

1. (5 pts) **Entrega 1 — Analisador Léxico (Flex)**

Projete e implemente um analisador léxico em **Flex** para a mini linguagem definida pelo grupo, observando obrigatoriamente os seguintes elementos:

- Declarações de tipos (`int`, `float`);
- Identificadores, números inteiros e decimais;
- Palavras-chave (`if`, `else`, `while`, `print`, `read`);
- Operadores relacionais, aritméticos e lógicos;
- Símbolos de pontuação (`;`, `,`, `(`, `)`, `{`, `}`).

PS: a linguagem deve permitir valores numéricos negativos; a forma de tratamento do sinal é uma decisão de projeto.

Critérios a serem avaliados:

- Além de reconhecer os *tokens*, deve:
 - estar organizado, com comentários explicativos;
 - imprimir o *token*, seu lexema e sua posição (linha e coluna);
 - detectar erros e reportá-los ao usuário informando a posição;
 - exibir a **tabela de símbolos** construída pelo analisador léxico; e
 - desconsiderar espaços em branco e comentários (linha única `//` e múltipla `/* */`).
- O **relatório** (conciso e objetivo)¹ deve:
 - discutir as **decisões de projeto**;
 - discutir as **dificuldades encontradas**;
 - apresentar uma **especificação resumida da linguagem proposta pelo grupo**, incluindo exemplos da sintaxe adotada para as construções exigidas neste enunciado;
 - apresentar dois **diagramas de transição** (DFAs) referentes a classes de *tokens*; e
 - incluir um arquivo de teste e sua saída (não precisa ser grande, porém completo).

¹ Os relatórios – para cada etapa – deverão ter, no máximo, **1 (uma) página**, em **duas colunas**, e deverão ser elaborados obrigatoriamente utilizando o formato padrão de artigos da **ACM**, com a classe `acmart` na opção `sigconf`.

2. (10 pts) Entrega 2 — Analisador Sintático (Bison)

Projete e implemente um analisador sintático em **Bison** para a linguagem proposta pelo grupo. A gramática deverá ser definida pelo próprio grupo e deve prover, obrigatoriamente:

- declarações de variáveis e tipos;
- instruções de atribuição;
- condicionais (**if/else**);
- laços (**while**);
- blocos e comandos compostos (**{ }**);
- chamadas de funções predefinidas da linguagem, incluindo operações de entrada/saída (**print, read**);
- expressões aritméticas, relacionais e lógicas, com precedência e associatividade definidas pelo grupo.

Critérios a serem avaliados:

- Correções apontadas pelo professor na Etapa 1.
- O analisador deve:
 - estar organizado, com comentários explicativos;
 - reconhecer a sintaxe de programas escritos na linguagem proposta;
 - compilar corretamente no Bison e rodar em conjunto com o analisador léxico da Etapa 1;
 - detectar e reportar **erros sintáticos** com mensagem clara indicando a posição (linha e coluna); e
 - resolver ambiguidades (e.g., *dangling else* e -).
- O **relatório** (conciso e objetivo)¹ deve:
 - discutir as **decisões de projeto**, incluindo ambiguidades e tratamento de erros;
 - comentar os principais resultados obtidos na análise da gramática;
 - discutir as **dificuldades encontradas**;
 - um artefato complementar – em um apêndice de no máximo uma página – com a gramática completa em BNF.

3. (25 pts) Entrega 3 — Análise Semântica e Geração de Código Intermediário

Implemente a **análise semântica** e a **geração de código de três endereços (IR)** para a mesma linguagem projetada e implementada nas Etapas 1 e 2. As decisões semânticas não determinadas explicitamente neste enunciado também constituem decisões de projeto do grupo e devem ser consistentes com a especificação da linguagem.

Critérios a serem avaliados:

- Correções apontadas pelo professor na Etapa 2.
- Manter consistência com as decisões de projeto adotadas nas etapas anteriores; eventuais alterações devem ser explicitamente descritas e justificadas.
- O compilador deve:
 - manter uma **tabela de símbolos** com **escopos aninhados** (abrir/fechar escopo em **{ }**), armazenando ao menos *identificador*, *tipo* e *nível de escopo*;
 - realizar **verificação de tipos** em:
 - * **atribuição** (lado direito compatível com o tipo da variável);
 - * **expressões aritméticas** (+ - * / aceitam **int** e **float** com promoção implícita de tipos², e % aceita apenas **int**);
 - * **relacionais** (== != < <= > >= tomam **int** ou **float** e produzem valores inteiros (0 ou 1)³);
 - * **lógicos** (! && || aceitam operandos numéricos, realizando conversão implícita para valores lógicos e produzem valores inteiros (0 ou 1));
 - * **unários** (- unário sobre **int** e **float**, ! sobre expressões numéricas);
 - * **if/while**: condição deve ser qualquer expressão numérica⁴.

²Quando um operando é **float**, o outro é promovido implicitamente para **float**.

³A linguagem não possui tipo booleano explícito. Valores inteiros são utilizados para representar resultados lógicos, onde 0 representa falso e qualquer valor diferente de 0 representa verdadeiro.

⁴A condição é avaliada como falsa quando igual a 0 e verdadeira caso contrário, seguindo a convenção de linguagens como C.

- **reportar erros semânticos** com posição, e.g., identificadores não declarados ou redeclarados no mesmo escopo.
- gerar código intermediário para todas as construções da linguagem;
 - * usar de temporários $(t_1, t_2, \dots, t_n)$ e rótulos $(L_1, L_2, \dots, L_m)$; e
 - * especificar e usar um conjunto mínimo de instruções de IR, incluindo a representação de chamadas às funções predefinidas da linguagem (e.g., `print` e `read`).
- O relatório (conciso e objetivo)¹ deve:
 - descrever a **estrutura da tabela de símbolos** e o **gerenciamento de escopos**;
 - apresentar uma **tabela de regras de tipagem** dos operadores/construções;
 - explicar a **estratégia de geração de IR** para `if/else` e `while`;
 - incluir um **programa de teste** e o **IR gerado** correspondente (saída do compilador);
 - apresentar pelo menos uma **Tradução Dirigida por Sintaxe (TDS)**; e
 - discutir **decisões de projeto** e **dificuldades encontradas**.

Entrega e execução

Para padronizar a correção e a execução dos trabalhos, deverão ser obrigatoriamente utilizados os seguintes nomes de arquivos:

- `lexer.l`: especificação do analisador léxico em Flex;
- `parser.y`: especificação do analisador sintático em Bison; e
- `compiler.sh`: *script* responsável pela compilação e execução do compilador.

O *script* `compiler.sh` deverá receber como argumento o arquivo-fonte a ser analisado e realizar **automaticamente todas as etapas necessárias** para gerar e executar o compilador, considerando um **ambiente Linux**. Por exemplo:

```
./compiler.sh programa.txt
```

Não deverão ser necessários comandos adicionais para compilar ou executar o trabalho. Isso será avaliado.

Política do Uso de IA

O uso de ferramentas de Inteligência Artificial (IA) é permitido como **apoio à aprendizagem**, mas não para substituir o trabalho intelectual e a implementação que devem ser realizados pelos próprios alunos.

Qualquer recurso utilizado no desenvolvimento do TP que **não esteja presente na apostila da disciplina ou nos códigos e materiais fornecidos pelo professor** deverá ter seu uso **previamente autorizado** pelo professor ou pelo monitor por meio do fórum “TP”, no Campus Virtual. O uso de recurso externo sem autorização implicará em **nota 0**.

Esta política não tem como objetivo penalizar o uso de novas ferramentas, mas garantir que os alunos **pesquisem, implementem e compreendam** as soluções desenvolvidas, sem delegar à IA ou a soluções prontas a aprendizagem esperada na disciplina.